

Delta Flow Volume Profile [UAlgo]

// This work is licensed under Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International

// <https://creativecommons.org/licenses/by-nc-sa/4.0/>

// © UAlgo

//@version=6

indicator("Delta Flow Volume Profile [UAlgo]", overlay=true, max_boxes_count=500, max_labels_count=500, max_lines_count=500)

int lookback = input<int>(200, "Lookback Period", minval=10)

int numBins = input<int>(50, "Number of Bins", minval=5, maxval=100)

float profileWidthPct = input<float>(30, "Profile Width %", minval=10, maxval=100)

int profileOffset = input<int>(5, "Profile Offset", minval=1)

color bullColor = input<color>(color.new(#00bcd4, 50), "Bullish Delta Color")

color bearColor = input<color>(color.new(#ff5252, 50), "Bearish Delta Color")

color pocColor = input<color>(color.new(#ffeb3b, 30), "POC Color")

type Bin

float top

float bottom

float bullVol

float bearVol

method totalVol(Bin this) =>

this.bullVol + this.bearVol

method deltaVol(Bin this) =>

this.bullVol - this.bearVol

method bullPct(Bin this) =>

tot = this.totalVol()

tot > 0 ? (this.bullVol / tot) * 100 : 0.0

method bearPct(Bin this) =>

tot = this.totalVol()

tot > 0 ? (this.bearVol / tot) * 100 : 0.0

```
float hh_global = ta.highest(high, lookback)
```

```
float ll_global = ta.lowest(low, lookback)
```

```
if barstate.islast
```

```
    float hh = hh_global
```

```
    float ll = ll_global
```

```
    float binSize = (hh - ll) / numBins
```

```
    array<Bin> bins = array.new<Bin>()
```

```
    for i = 0 to numBins - 1
```

```
        float bTop = hh - (i × binSize)
```

```
        float bBot = hh - ((i + 1) × binSize)
```

```
        bins.push(Bin.new(bTop, bBot, 0.0, 0.0))
```

```
    for i = 0 to lookback - 1
```

```
        float bHigh = high[i]
```

```
        float bLow = low[i]
```

```
        float bClose = close[i]
```

```
        float bOpen = open[i]
```

```
        float bVol = nz(volume[i])
```

```
    if bVol > 0
```

```
        for j = 0 to numBins - 1
```

```
            Bin b = bins.get(j)
```

```
            float overlapTop = math.min(bHigh, b.top)
```

```
            float overlapBottom = math.max(bLow, b.bottom)
```

```
            if overlapTop >= overlapBottom
```

```
                float rangeSize = bHigh - bLow
```

```
                float allocatedVol = 0.0
```

```
                if rangeSize > 0
```

```
                    allocatedVol := bVol * ((overlapTop - overlapBottom) / rangeSize)
```

```
                else
```

```
                    if bHigh <= b.top and (bHigh > b.bottom or (j == numBins - 1 and bHigh >=
b.bottom))
```

```
                        allocatedVol := bVol
```

```
    if allocatedVol > 0
        if bClose >= bOpen
            bxbullVol += allocatedVol
        else
            bxbearVol += allocatedVol
```

```
float maxVol = 0.0
int pocIdx = -1
for i = 0 to numBins - 1
    float bTotVol = binsxget(i)×totalVol()
    if bTotVol > maxVol
        maxVol := bTotVol
        pocIdx := i
```

```
int lastBarTime = time
int barTimeDiff = time - time[1]
```

```
if na(barTimeDiff) or barTimeDiff <= 0
    barTimeDiff := 1000 * 60 * 60
```

```
int maxBoxWidthBars = math×floor(lookback × (profileWidthPct / 100.0))
int leftBaseTime = lastBarTime + (profileOffset × barTimeDiff)
int maxRightTime = leftBaseTime + (maxBoxWidthBars × barTimeDiff)
```

```
for i = 0 to numBins - 1
    Bin b = binsxget(i)
    float bTotVol = b×totalVol()

    if bTotVol > 0
        float bullRatio = bxbullVol / maxVol
        float bearRatio = bxbearVol / maxVol
```

```
int bullWidthBars = math×round(bullRatio × maxBoxWidthBars)
int bearWidthBars = math×round(bearRatio × maxBoxWidthBars)
```

```
if b.bullVol > 0 and bullWidthBars == 0
    bullWidthBars := 1
```

```

if b.bearVol > 0 and bearWidthBars == 0
    bearWidthBars := 1

int rightBullTime = leftBaseTime + (bullWidthBars × barTimeDiff)
int rightBearTime = rightBullTime + (bearWidthBars × barTimeDiff)

float boxGap = (b×top - b×bottom) × 0.1
float drawTop = b×top - boxGap
float drawBottom = b×bottom + boxGap

if i == pocIdx
    box.new(left=leftBaseTime, top=drawTop, right=maxRightTime, bottom=drawBottom,
xloc=xloc.bar_time,
        border_color=na, bgcolor=color.new(pocColor, 85))

if bullWidthBars > 0
    box.new(left=leftBaseTime, top=drawTop, right=rightBullTime, bottom=drawBottom,
xloc=xloc.bar_time,
        border_color=na, bgcolor=bullColor)

if bearWidthBars > 0
    box.new(left=rightBullTime, top=drawTop, right=rightBearTime, bottom=drawBottom,
xloc=xloc.bar_time,
        border_color=na, bgcolor=bearColor)

if bTotVol > (maxVol * 0.15)
    string bullStr = str×tostring(math×round(b×bullPct())) + "%"
    string bearStr = str×tostring(math×round(b×bearPct())) + "%"
    string labelText = bullStr + " | " + bearStr

if i == pocIdx
    labelText := "POC: " + labelText

int labelXPos = leftBaseTime + ((rightBearTime - leftBaseTime) / 2)

color labelTextColor = color.white
if i == pocIdx
    labelTextColor := color.new(#fbc02d, 0)

```

```
label.new(x=labelXPos, y=(bxtop + bxbottom) / 2, text=labelText, xloc=xloc.bar_time,  
yloc=yloc.price,  
color=na, style=label.style_label_center, textcolor=labelTextColor, size=size.small)
```